

UTS
PENGOLAHAN CITRA



NAMA : Raihan Daris Ramadhan
NIM : 202431033
KELAS : E
DOSEN : Rizqia Cahyaningtyas, ST., M.Kom
NO.PC : -
ASISTEN : 1. Joshua Indra Pratama
2. Davina Najwa Ermawan
3. Izzat Islami Kagapi
4. Sakura Amastasya Salsabila Setiyanto

INSTITUT TEKNOLOGI PLN
TEKNIK INFORMATIKA
2025/2026

DAFTAR ISI

DAFTAR ISI	2
BAB I	3
PENDAHULUAN	3
1.1 Rumusan Masalah	3
1.2 Tujuan Masalah	3
1.3 Manfaat Masalah	3
BAB II	4
LANDASAN TEORI	4
2.1 Pengolahan Citra Digital	4
2.2 Python	4
2.3 OpenCV	4
2.4 Model Warna RGB	5
2.5 Model Warna HSV	5
2.6 Thresholding	5
2.7 Histogram	6
2.8 Grayscale	6
2.9 Brightness	6
2.10 Contrast	6
2.11 Efek Backlight pada Citra	7
BAB III	8
HASIL	8
3.1 Gambar Asli – Citra Tulisan Nama	8
3.2 Hasil Konversi HSV	8
3.3 Hasil Deteksi Warna Merah	9
3.4 Hasil Deteksi Warna Hijau	10
3.5 Hasil Deteksi Warna Biru	10
3.6 Histogram RGB	11
3.7 Nilai Threshold HSV	12
3.8 Gambar Asli Backlight	13
3.9 Hasil Grayscale	14
3.10 Hasil Brightness	15
3.11 Hasil Contrast	16
3.12 Hasil Brightness dan Contrast	17
3.13 Screenshot Detail Foto (Bukti Pengambilan Citra)	17
BAB IV	19
PENUTUP	19
DAFTAR PUSTAKA	20

BAB I

PENDAHULUAN

1.1 Rumusan Masalah

1. Bagaimana cara mendeteksi warna merah, hijau, dan biru pada sebuah citra digital menggunakan Python dan OpenCV dengan pendekatan ruang warna HSV?
2. Bagaimana cara membuat dan menganalisis histogram distribusi warna RGB pada citra digital?
3. Bagaimana menentukan nilai ambang batas (threshold) HSV yang optimal untuk setiap kategori warna?
4. Bagaimana cara memperbaiki kualitas gambar yang mengalami efek backlight melalui operasi pixel (grayscale, brightness, dan contrast)?

1.2 Tujuan Masalah

1. Mengetahui dan memahami cara deteksi warna pada citra digital menggunakan pendekatan HSV thresholding.
2. Memahami cara membuat dan membaca histogram warna RGB untuk menganalisis distribusi intensitas pixel.
3. Menentukan rentang nilai threshold HSV yang tepat melalui eksperimen untuk memisahkan warna merah, hijau, dan biru.
4. Menerapkan teknik peningkatan brightness dan contrast untuk memperbaiki citra yang terdampak efek backlight.

1.3 Manfaat Masalah

1. Menambah pemahaman mendalam mengenai konsep dan teknik pengolahan citra digital, khususnya segmentasi berbasis warna.
2. Memahami implementasi praktis Python dan OpenCV dalam pemrosesan gambar secara nyata.
3. Mengetahui cara kerja dan perbedaan antara model warna RGB dan HSV dalam konteks deteksi objek berwarna.
4. Mengetahui cara memperbaiki kualitas gambar yang gelap akibat kondisi pencahayaan backlight menggunakan operasi pixel sederhana.
5. Memberikan pengalaman langsung dalam menganalisis hasil pengolahan citra secara visual dan kuantitatif melalui histogram.

BAB II

LANDASAN TEORI

2.1 Pengolahan Citra Digital

Pengolahan citra digital (digital image processing) adalah proses memanipulasi dan menganalisis gambar menggunakan komputer dengan tujuan untuk meningkatkan kualitas visual, mengekstraksi informasi penting, atau mempersiapkan gambar untuk analisis lebih lanjut. Sebuah citra digital tersusun atas matriks piksel (picture element) dua dimensi, di mana setiap piksel memiliki nilai numerik yang merepresentasikan intensitas warna pada titik tersebut (Farhan & Wibowo, 2022).

Pengolahan citra digital telah berkembang pesat dan kini diterapkan di berbagai bidang: sistem pengenalan wajah pada kamera keamanan, deteksi penyakit pada citra medis (CT scan, MRI), inspeksi kualitas produk di industri, klasifikasi tanaman dalam pertanian presisi, serta pengembangan kendaraan otonom. Secara umum, tahapan pengolahan citra meliputi akuisisi citra, pra pemrosesan, segmentasi, ekstraksi fitur, dan interpretasi (Nugroho & Santoso, 2023).

2.2 Python

Python adalah bahasa pemrograman tingkat tinggi yang bersifat interpreted dan multi paradigma, pertama kali dikembangkan oleh Guido van Rossum pada tahun 1991. Python sangat populer di kalangan peneliti dan praktisi pengolahan citra karena sintaksisnya yang bersih dan mudah dipahami, didukung oleh ekosistem library yang sangat kaya. Dalam konteks computer vision dan image processing, Python menjadi bahasa pilihan utama karena dukungan library seperti OpenCV, NumPy, dan Matplotlib yang terintegrasi dengan baik (Adriansyah & Kurniati, 2022).

Pada praktikum ini, Python digunakan untuk mengimplementasikan seluruh pipeline pengolahan citra, mulai dari pembacaan gambar, konversi ruang warna, deteksi objek berbasis warna, visualisasi histogram, hingga perbaikan kualitas citra backlight.

2.3 OpenCV

OpenCV (Open Source Computer Vision Library) adalah library open source yang dikembangkan oleh Intel dan dirilis pertama kali pada tahun 1999. Saat ini OpenCV mendukung lebih dari 2.500 algoritma computer vision yang teroptimasi, tersedia untuk bahasa C++, Python, Java, dan MATLAB. OpenCV secara aktif digunakan dalam penelitian akademik maupun aplikasi industri di seluruh dunia (Wahyudi & Pramono, 2021).

Dalam praktikum UTS ini, OpenCV digunakan untuk:

- Membaca dan menampilkan gambar (cv2.imread, cv2.imshow)
- Konversi ruang warna BGR ke RGB dan BGR ke HSV (cv2.cvtColor)

- Segmentasi berbasis rentang warna (cv2.inRange)
- Operasi masking piksel (cv2.bitwise_and)
- Perhitungan histogram warna (cv2.calcHist)
- Peningkatan brightness dan contrast (cv2.convertScaleAbs)

2.4 Model Warna RGB

RGB adalah model warna aditif yang menyatakan setiap warna sebagai kombinasi tiga komponen: Red (merah), Green (hijau), dan Blue (biru). Nilai setiap komponen berkisar dari 0 hingga 255, sehingga terdapat $256^3 = 16.777.216$ kemungkinan kombinasi warna. Model ini paling umum digunakan pada layar monitor, kamera digital, dan sistem pencitraan elektronik (Hidayatullah, 2021).

Contoh representasi warna dalam RGB:

- Merah murni : (255, 0, 0)
- Hijau murni : (0, 255, 0)
- Biru murni : (0, 0, 255)
- Putih : (255, 255, 255)
- Hitam : (0, 0, 0)

Meskipun RGB mudah dimengerti, model ini kurang efektif untuk deteksi warna dalam kondisi pencahayaan yang bervariasi, karena ketiga komponen berubah secara bersamaan saat tingkat kecerahan berubah.

2.5 Model Warna HSV

HSV (Hue, Saturation, Value) adalah model warna yang dirancang agar lebih sesuai dengan cara manusia mempersepsi warna. Komponen Hue merepresentasikan jenis warna dalam sudut lingkaran $0^\circ - 360^\circ$ (dalam OpenCV, rentang Hue menggunakan skala 0 - 179 untuk efisiensi penyimpanan 8 bit). Saturation menggambarkan tingkat kejenuhan atau kemurnian warna (0 = abu abu, 255 = warna penuh), sedangkan Value menunjukkan tingkat kecerahan (0 = hitam, 255 = warna terang) (Ramadhan & Kusuma, 2023).

Keunggulan HSV dalam deteksi warna adalah kemampuannya untuk mengisolasi warna tertentu menggunakan satu rentang Hue, terlepas dari perubahan kecerahan dan saturasi. Hal ini menjadikan HSV jauh lebih robust dibandingkan RGB ketika digunakan dalam kondisi pencahayaan yang tidak seragam.

2.6 Thresholding

Thresholding adalah teknik segmentasi citra yang memisahkan region objek dari latar belakang berdasarkan nilai ambang batas intensitas pixel tertentu. Dalam pengolahan warna, thresholding diterapkan pada ruang warna HSV menggunakan fungsi cv2.inRange() yang menghasilkan citra biner (mask): piksel yang masuk dalam rentang threshold bernilai 255 (putih/aktif), sementara piksel di luar rentang bernilai 0 (hitam/tidak aktif) (Setiawan & Hendra, 2022).

Warna merah dalam HSV memiliki karakteristik khusus karena nilai Hue-nya berada di dua ujung rentang (mendekati 0° dan mendekati $360^\circ/179$ dalam skala OpenCV), sehingga diperlukan dua mask terpisah yang kemudian digabungkan dengan operasi penjumlahan atau bitwise OR.

2.7 Histogram

Histogram citra adalah representasi grafis yang menunjukkan distribusi frekuensi nilai intensitas piksel dalam sebuah gambar. Sumbu horizontal (X) merepresentasikan nilai intensitas piksel (0 – 255), sedangkan sumbu vertikal (Y) menunjukkan jumlah piksel yang memiliki intensitas tersebut. Histogram memberikan informasi kuantitatif mengenai karakteristik global citra, termasuk tingkat kecerahan, kontras, serta dominasi warna tertentu (Nugroho & Santoso, 2023).

Pada gambar dengan banyak piksel putih (latar belakang kertas), histogram akan menunjukkan puncak tinggi di nilai intensitas mendekati 255. Sebaliknya, pada gambar yang gelap, distribusi piksel akan terpusat di nilai intensitas rendah (mendekati 0).

2.8 Grayscale

Grayscale adalah representasi citra yang hanya menggunakan satu kanal intensitas abu - abu, tanpa informasi warna. Konversi dari BGR ke grayscale dilakukan dengan menghitung rata - rata tertimbang dari ketiga komponen warna menggunakan formula: $Y = 0.114 \cdot B + 0.587 \cdot G + 0.299 \cdot R$, yang sesuai dengan sensitivitas mata manusia terhadap warna (Hidayatullah, 2021).

Penggunaan grayscale dalam perbaikan gambar backlight memberikan keuntungan signifikan: jumlah data yang harus diproses berkurang menjadi sepertiga, operasi matematika menjadi lebih sederhana, dan hasil peningkatan brightness/contrast lebih mudah dikontrol karena hanya ada satu kanal yang perlu dimanipulasi.

2.9 Brightness

Brightness atau kecerahan mengacu pada persepsi visual tentang seberapa terang suatu gambar. Dalam pengolahan citra digital, peningkatan brightness dilakukan dengan menambahkan nilai konstan (β) pada setiap piksel: $\text{Output} = \alpha \times \text{Input} + \beta$. Dengan menetapkan $\alpha = 1$ dan $\beta > 0$, seluruh piksel akan bergeser ke nilai yang lebih tinggi (lebih terang) tanpa mengubah kontras relatif antar piksel (Wahyudi & Pramono, 2021).

Pada kasus gambar backlight, peningkatan brightness sangat diperlukan agar area subjek yang sebelumnya tampak sebagai siluet gelap dapat terlihat lebih jelas dan detailnya dapat diamati.

2.10 Contrast

Contrast adalah perbedaan antara nilai piksel terang dan gelap dalam sebuah gambar. Peningkatan kontras dilakukan dengan memperbesar rentang dinamis nilai piksel menggunakan pengali (α): $\text{Output} = \alpha \times \text{Input} + \beta$, di mana $\alpha > 1$ akan

meningkatkan kontras dan $\alpha < 1$ akan mengurangnya. Nilai $\beta = 0$ digunakan ketika hanya kontras yang ingin diubah tanpa pergeseran kecerahan (Adriansyah & Kurniati, 2022).

Peningkatan kontras mempertajam perbedaan antara tepi objek dan latar belakang, sehingga detail - detail halus seperti kontur wajah, tekstur kain, atau garis objek menjadi lebih tampak jelas dibandingkan citra aslinya.

2.11 Efek Backlight pada Citra

Backlight adalah kondisi fotografis di mana sumber cahaya utama berada di belakang objek yang difoto. Akibatnya, sensor kamera akan melakukan exposure berdasarkan area latar belakang yang sangat terang, sehingga area objek utama yang berada di depan sumber cahaya menjadi sangat kurang terekspos dan tampak gelap atau sebagai siluet (Setiawan & Hendra, 2022).

Secara teknis, gambar backlight memiliki histogram yang bimodal puncak tinggi di nilai rendah (area gelap/objek) dan puncak tinggi di nilai tinggi (area terang/latar belakang) dengan lembah di tengahnya. Teknik perbaikan yang efektif mencakup histogram equalization, gamma correction, atau kombinasi peningkatan brightness dan contrast seperti yang diterapkan pada praktikum ini.

BAB III

HASIL

3.1 Gambar Asli – Citra Tulisan Nama

```
In [2]: M img = cv2.imread("6068673310922313520.jpg")
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(8, 4))
plt.imshow(img_rgb)
plt.title("Foto Asli - Tulisan Nama")
plt.axis("off")
plt.tight_layout()
plt.show()
```

Foto Asli - Tulisan Nama



Gambar 3.1 – Citra tulisan nama "Raihan Daris Ramadhan" dalam tiga warna

Gambar asli yang digunakan pada soal nomor 1 adalah citra tulisan tangan nama "Raihan Daris Ramadhan" menggunakan spidol warna merah (untuk kata "Raihan"), hijau (untuk kata "Daris"), dan biru (untuk kata "Ramadhan") di atas kertas putih. Pemilihan latar belakang putih disengaja agar deteksi warna menjadi lebih akurat karena kontras antara tinta berwarna dan latar putih sangat tinggi. Gambar diambil menggunakan kamera Samsung SM-A235F dengan resolusi 2794×1002 piksel.

3.2 Hasil Konversi HSV

```
In [3]: M hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

plt.figure(figsize=(8, 4))
plt.imshow(cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB))
plt.title("Gambar HSV")
plt.axis("off")
plt.tight_layout()
plt.show()

print("✓ Gambar berhasil dikonversi ke HSV")
```

Gambar HSV



✓ Gambar berhasil dikonversi ke HSV

Gambar 3.2 – Citra tulisan nama "Raihan Daris Ramadhan" dalam HSV

Sebelum proses deteksi warna dilakukan, gambar perlu dikonversi terlebih dahulu dari ruang warna BGR (format standar OpenCV) ke ruang warna HSV menggunakan fungsi `cv2.cvtColor(img, cv2.COLOR_BGR2HSV)`. Langkah ini penting karena nilai Hue dalam HSV secara langsung merepresentasikan jenis warna, terlepas dari perubahan kecerahan atau saturasi.

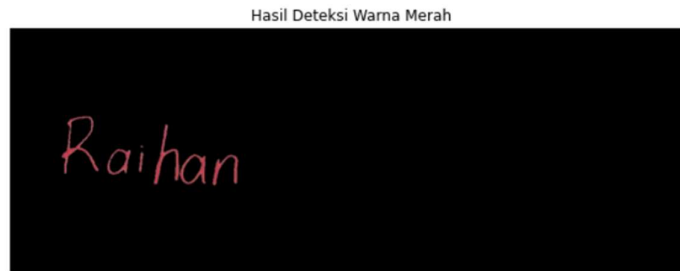
3.3 Hasil Deteksi Warna Merah

```
In [4]: merah_bawah1 = np.array([0, 100, 100])
merah_atas1 = np.array([10, 255, 255])
merah_bawah2 = np.array([160, 100, 100])
merah_atas2 = np.array([179, 255, 255])

mask1 = cv2.inRange(hsv, merah_bawah1, merah_atas1)
mask2 = cv2.inRange(hsv, merah_bawah2, merah_atas2)
mask_merah = mask1 + mask2

hasil_merah = cv2.bitwise_and(img_rgb, img_rgb, mask=mask_merah)

plt.figure(figsize=(8, 4))
plt.imshow(hasil_merah)
plt.title("Hasil Deteksi Warna Merah")
plt.axis("off")
plt.tight_layout()
plt.show()
```



Gambar 3.3 – Deteksi Warna Merah

Deteksi warna merah memerlukan dua rentang threshold HSV karena nilai Hue untuk warna merah dalam OpenCV (skala 0 – 179) berada di dua ujung lingkaran warna: rentang bawah 0 – 10 dan rentang atas 160 – 179. Kedua mask yang dihasilkan kemudian digabungkan menggunakan operasi penjumlahan (`mask1 + mask2`), lalu diterapkan ke citra asli menggunakan `cv2.bitwise_and()`.

Hasil yang diperoleh menunjukkan kata "Raihan" (berwarna merah) berhasil diisolasi dari kedua kata lainnya. Area latar belakang putih dan tulisan berwarna lain berubah menjadi hitam karena nilainya berada di luar rentang threshold merah yang telah ditetapkan. Sedikit noise dapat terlihat pada beberapa piksel, namun secara keseluruhan isolasi warna merah berjalan dengan baik.

3.4 Hasil Deteksi Warna Hijau

```
In [5]: hijau_bawah = np.array([40, 50, 50])
        hijau_atas = np.array([90, 255, 255])

        mask_hijau = cv2.inRange(hsv, hijau_bawah, hijau_atas)
        hasil_hijau = cv2.bitwise_and(img_rgb, img_rgb, mask=mask_hijau)

        plt.figure(figsize=(8, 4))
        plt.imshow(hasil_hijau)
        plt.title("Hasil Deteksi Warna Hijau")
        plt.axis("off")
        plt.tight_layout()
        plt.show()
```



Gambar 3.4 – Deteksi Warna Hijau

Warna hijau memiliki rentang Hue yang lebih sederhana dibandingkan merah karena tidak mengalami wrap around. Rentang threshold yang digunakan adalah Hue 40 – 90, Saturation 50 – 255, dan Value 50 – 255. Rentang ini cukup lebar untuk menangkap variasi warna hijau spidol yang mungkin memiliki sedikit perbedaan nuansa di setiap bagian tulisan akibat variasi tekanan atau sudut pengambilan gambar.

Hasil deteksi menunjukkan kata "Daris" (berwarna hijau) berhasil terpisah dari dua kata lainnya dengan cukup jelas. Background tidak banyak ikut terdeteksi, yang mengindikasikan bahwa rentang threshold yang dipilih sudah cukup spesifik untuk warna hijau spidol ini.

3.5 Hasil Deteksi Warna Biru

```
In [9]: biru_bawah = np.array([90, 50, 50])
        biru_atas = np.array([140, 255, 255])

        mask_biru = cv2.inRange(hsv, biru_bawah, biru_atas)
        hasil_biru = cv2.bitwise_and(img_rgb, img_rgb, mask=mask_biru)

        plt.figure(figsize=(8, 4))
        plt.imshow(hasil_biru)
        plt.title("Hasil Deteksi Warna Biru")
        plt.axis("off")
        plt.tight_layout()
        plt.show()
```



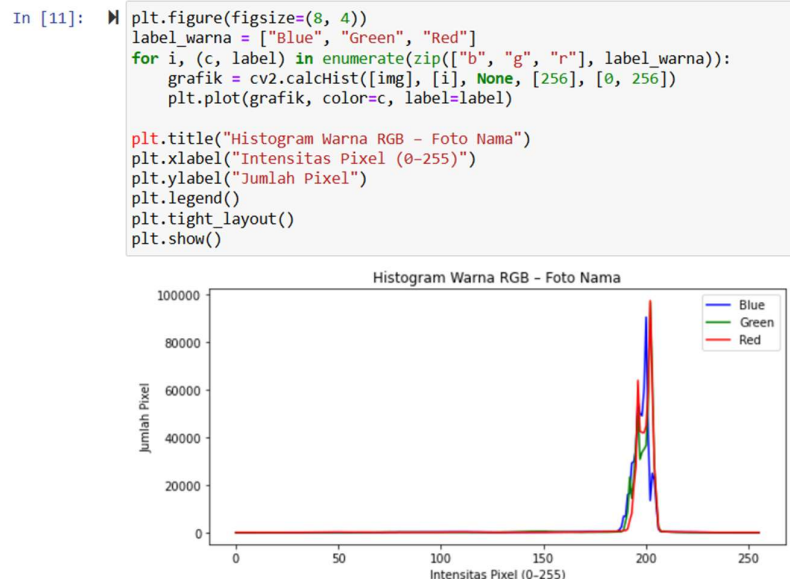
Gambar 3.5 – Deteksi Warna Biru

Warna biru menggunakan rentang threshold Hue 90 – 140, Saturation 50 – 255, dan Value 50 – 255. Rentang ini mencakup berbagai nuansa biru, dari biru tua hingga biru muda, yang biasanya ditemukan pada spidol warna biru standar. Kata "Ramadhan" yang ditulis dengan spidol biru berhasil dideteksi dan diisolasi.

Hasil isolasi warna biru menampilkan kata "Ramadhan" dengan latar hitam, sementara kedua kata lain (merah dan hijau) tidak tampak. Keberhasilan deteksi ini memvalidasi bahwa nilai threshold biru Hue 100–140 sesuai untuk karakteristik warna biru spidol yang digunakan. Beberapa area mungkin memiliki intensitas berbeda akibat pantulan cahaya pada permukaan tinta, namun hal ini masih dalam batas toleransi yang dapat diterima.

3.6 Histogram RGB

Histogram RGB



Gambar 3.6 – Histogram RGB

Histogram RGB menampilkan distribusi nilai intensitas piksel untuk ketiga kanal warna (Red, Green, Blue) secara bersamaan dalam satu grafik. Dari hasil histogram yang diperoleh, terlihat puncak yang sangat tinggi pada nilai intensitas mendekati 200 – 255 untuk semua kanal. Hal ini wajar karena sebagian besar area gambar merupakan latar belakang kertas putih yang memiliki nilai piksel tinggi di semua kanal RGB.

Di sisi nilai rendah (0 – 100), terdapat distribusi piksel yang lebih kecil yang merepresentasikan area tulisan berwarna. Kanal merah, hijau, dan biru masing-masing menunjukkan pola distribusi yang berbeda di area ini, mencerminkan kontribusi warna tinta terhadap komposisi pixel keseluruhan. Analisis histogram ini mengkonfirmasi bahwa gambar didominasi oleh warna putih (latar belakang) dengan sebaran warna tinta yang relatif sedikit secara proporsi.

3.7 Nilai Threshold HSV

Nilai Threshold HSV

```
In [12]: print("\n=== NILAI THRESHOLD HSV ===")
print(f"Merah : Hue 0-10 DAN 160-179 | Sat 100-255 | Val 100-255")
print(f"Hijau : Hue 40-90 | Sat 50-255 | Val 50-255")
print(f"Biru : Hue 90-140 | Sat 50-255 | Val 50-255")

# -- 9. Tampil Threshold (NONE / BLUE / RED-BLUE / ALL) --
mask_none = np.zeros(img.shape[:2], dtype=np.uint8)
mask_red_blue = mask_merah + mask_biru
mask_all = mask_merah + mask_hijau + mask_biru

hasil_none = cv2.bitwise_and(img_rgb, img_rgb, mask=mask_none)
hasil_red_blue = cv2.bitwise_and(img_rgb, img_rgb, mask=mask_red_blue)
hasil_all = cv2.bitwise_and(img_rgb, img_rgb, mask=mask_all)

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
axes[0,0].imshow(hasil_none); axes[0,0].set_title("NONE"); axes[0,0].axis("off")
axes[0,1].imshow(hasil_biru); axes[0,1].set_title("BLUE"); axes[0,1].axis("off")
axes[1,0].imshow(hasil_red_blue); axes[1,0].set_title("RED-BLUE"); axes[1,0].axis("off")
axes[1,1].imshow(hasil_all); axes[1,1].set_title("RED-GREEN-BLUE"); axes[1,1].axis("off")
plt.suptitle("Kategori Warna Berdasarkan Threshold HSV", fontsize=13, fontweight="bold")
plt.tight_layout()
plt.show()

=== NILAI THRESHOLD HSV ===
Merah : Hue 0-10 DAN 160-179 | Sat 100-255 | Val 100-255
Hijau : Hue 40-90 | Sat 50-255 | Val 50-255
Biru : Hue 90-140 | Sat 50-255 | Val 50-255
```

Kategori Warna Berdasarkan Threshold HSV



Gambar 3.7 – Nilai Threshold HSV

Penentuan nilai threshold dilakukan melalui serangkaian percobaan (trial and error) dengan mengamati hasil mask yang dihasilkan. Nilai threshold yang terlalu sempit akan melewatkan piksel piksel warna yang seharusnya terdeteksi (false negative), sementara nilai yang terlalu lebar akan menangkap piksel yang tidak diinginkan dari area lain (false positive/noise).

Warna merah memerlukan dua rentang karena distribusi Huenya dalam sistem OpenCV (0 – 179) tidak kontinu, merah berada di dekat 0 dan di dekat 179 sekaligus. Nilai Saturation dan Value minimum sebesar 100 dipilih untuk merah agar hanya piksel dengan warna jenuh dan terang yang terdeteksi. Untuk hijau dan biru, nilai minimum 50 digunakan karena warna spidol tersebut sedikit lebih pastel dibandingkan merah.

3.8 Gambar Asli Backlight

```
In [13]: img2 = cv2.imread("6068673310922313512.jpg")  
img2_rgb = cv2.cvtColor(img2, cv2.COLOR_BGR2RGB)  
  
plt.figure(figsize=(5, 7))  
plt.imshow(img2_rgb)  
plt.title("Gambar Asli - Backlight")  
plt.axis("off")  
plt.tight_layout()  
plt.show()
```



Gambar 3.8 – Foto asli backlight Raihan Daris Ramadhan

Foto backlight diambil dengan posisi subjek (Raihan Daris Ramadhan) menghadap kamera sementara cahaya alami dari jendela berada tepat di belakangnya. Kamera Samsung SM-A235F secara otomatis melakukan exposure compensation berdasarkan area terang di latar belakang (langit dan rumah di luar jendela), sehingga subjek utama menjadi sangat kurang terekspos dan tampak sebagai siluet gelap.

Foto ini diambil pada hari Jumat, 15 Mei 2026 pukul 09.48 WIB di Jl. Prof. Dr. Hamka No.38, Larangan, Kota Tangerang. Kondisi pencahayaan pagi hari dari jendela.

3.9 Hasil Grayscale

Konversi Ke Gray

```
In [14]: abu = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(5, 7))
plt.imshow(abu, cmap="gray")
plt.title("Gambar Gray")
plt.axis("off")
plt.tight_layout()
plt.show()
```



Gambar 3.9 – Hasil Gray Scale

Konversi ke grayscale dilakukan menggunakan `cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)`. Hasilnya adalah gambar satu kanal yang menampilkan intensitas abu-abu tanpa informasi warna. Pada gambar grayscale, area latar belakang (jendela dan lingkungan luar) tampak sangat terang (nilai piksel tinggi) sementara area siluet subjek tampak sangat gelap (nilai piksel rendah).

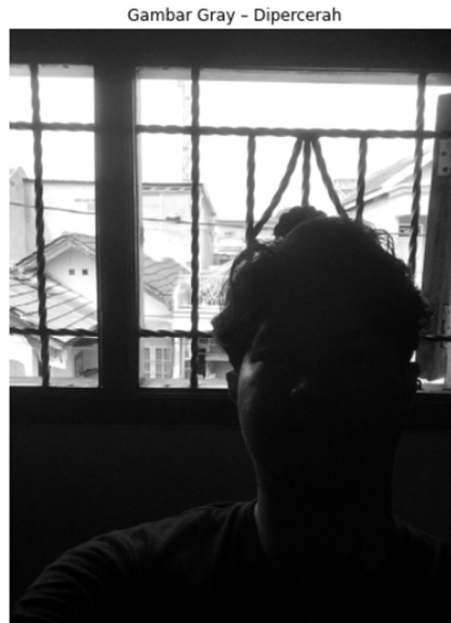
Alasan utama menggunakan grayscale sebelum operasi brightness/contrast adalah penyederhanaan perhitungan: dengan satu kanal, operasi konvolusi matriks berkurang dari tiga kanal menjadi satu, sehingga proses lebih efisien. Selain itu, hasil akhirnya lebih mudah dievaluasi secara visual karena perubahan kecerahan tidak terdistorsi oleh interaksi antar kanal warna.

3.10 Hasil Brightness

Hasil Brightness

```
In [17]: terang = cv2.convertScaleAbs(abu, alpha=1, beta=35)

plt.figure(figsize=(5, 7))
plt.imshow(terang, cmap="gray")
plt.title("Gambar Gray - Dipercerah")
plt.axis("off")
plt.tight_layout()
plt.show()
```



Gambar 3.10 – Hasil brightness

Peningkatan brightness dilakukan dengan formula: $\text{Output} = 1 \times \text{Input} + 35$ ($\alpha=1$, $\beta=35$). Parameter $\beta=35$ dipilih setelah beberapa kali percobaan, nilai yang terlalu kecil (misalnya 10) tidak memberikan perubahan signifikan, sedangkan nilai yang terlalu besar (misalnya 100) menyebabkan area latar belakang menjadi terlalu jenuh (overexposed) dan detail di area terang hilang.

Setelah peningkatan brightness, area siluet subjek terlihat sedikit lebih terang dibandingkan gambar aslinya. Beberapa detail di area gelap (seperti kontur kepala dan bahu) mulai dapat dibedakan. Namun, karena α tetap 1, kontras antara area terang dan gelap belum meningkat secara signifikan—hal ini akan diatasi pada langkah berikutnya.

3.11 Hasil Contrast

Peningkatan Kontras

```
In [18]: M kontras = cv2.convertScaleAbs(abu, alpha=1.7, beta=0)

plt.figure(figsize=(5, 7))
plt.imshow(kontras, cmap="gray")
plt.title("Gambar Gray - Diperkontras")
plt.axis("off")
plt.tight_layout()
plt.show()
```



Gambar 3.11 – Hasil Peningkatan Contrast

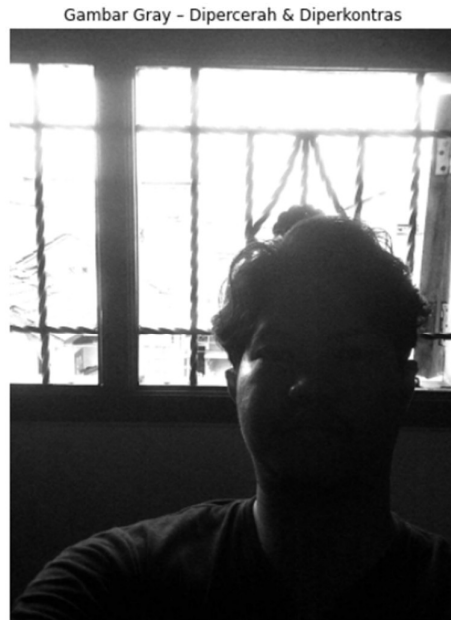
Peningkatan kontras dilakukan dengan formula: $\text{Output} = 1.7 \times \text{Input} + 0$ ($\alpha=1.7$, $\beta=0$). Nilai $\alpha > 1$ memperlebar jarak antara nilai piksel terang dan gelap. Dari hasil yang terlihat, area jendela dan latar belakang luar ruangan menjadi jauh lebih terang dan hampir putih penuh, sementara area siluet subjek tetap dominan gelap.

Hal ini terjadi karena piksel yang sudah memiliki nilai tinggi (background) semakin meningkat drastis, sedangkan piksel dengan nilai sangat rendah (siluet wajah) ketika dikalikan 1.7 tetap menghasilkan nilai yang masih tergolong gelap. Peningkatan kontras tanpa penambahan nilai beta tidak mampu mengangkat area gelap yang nilainya mendekati nol, sehingga detail wajah subjek belum dapat terlihat jelas.

3.12 Hasil Brightness dan Contrast

Kombinasi Brightness + Contrast

```
In [19]: M hasil = cv2.convertScaleAbs(abu, alpha=2, beta=30)
plt.figure(figsize=(5, 7))
plt.imshow(hasil, cmap="gray")
plt.title("Gambar Gray - Diperceh & Diperkontras")
plt.axis("off")
plt.tight_layout()
plt.show()
```



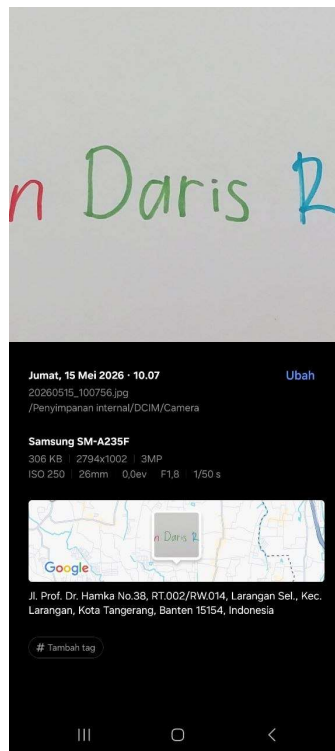
Gambar 3.12 – Hasil Peningkatan Contrast + Brightness

Kombinasi brightness dan contrast dilakukan dengan formula: $\text{Output} = 2 \times \text{Input} + 30$ ($\alpha=2$, $\beta=30$). Dibandingkan hanya menggunakan kontras saja, penambahan nilai $\beta=30$ memberikan efek pergeseran kecerahan pada seluruh piksel secara merata. Dari hasil yang diperoleh, outline kepala dan rambut subjek mulai sedikit lebih dapat dibedakan dari sebelumnya. Namun demikian, wajah subjek secara keseluruhan masih tampak sebagai siluet gelap karena nilai piksel awalnya yang sangat rendah, meskipun dikalikan 2 dan ditambah 30, hasilnya tetap belum cukup terang untuk memunculkan detail wajah secara signifikan.

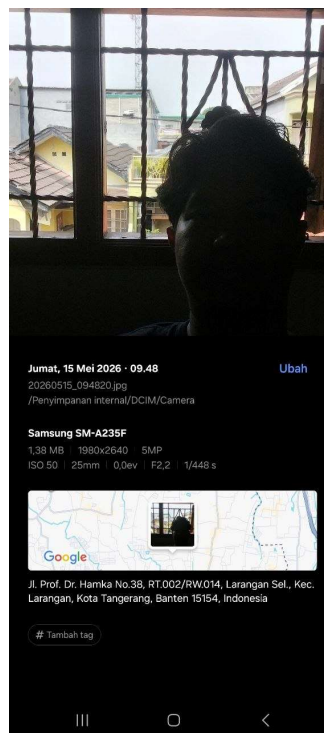
Di sisi lain, area latar belakang (jendela) mengalami color burn yang cukup terlihat akibat nilai pikselnya yang sudah tinggi semakin diperbesar oleh $\alpha=2$. Kondisi ini menunjukkan bahwa teknik linear brightness/contrast adjustment memiliki keterbatasan dalam menangani gambar backlight ekstrem, teknik yang lebih adaptif seperti histogram equalization atau CLAHE akan lebih efektif untuk kasus seperti ini.

3.13 Screenshot Detail Foto (Bukti Pengambilan Citra)

Berikut adalah bukti screenshot detail foto yang digunakan dalam praktikum ini, menunjukkan tanggal, waktu, perangkat, dan lokasi pengambilan gambar.

Detail Foto Tulisan Nama:

Gambar 3.13a – Metadata foto tulisan nama (Samsung SM-A235F, 15 Mei 2026, Tangerang)

Detail Foto Backlight:

Gambar 3.13b – Metadata foto backlight (Samsung SM-A235F, 15 Mei 2026, Tangerang)

BAB IV

PENUTUP

Berdasarkan hasil pengerjaan UTS Pengolahan Citra Digital ini, dapat ditarik beberapa simpulan sebagai berikut:

1. Deteksi warna merah, hijau, dan biru pada citra tulisan nama berhasil dilakukan menggunakan pendekatan konversi ruang warna HSV dan thresholding dengan `cv2.inRange()`. Masing-masing warna dapat diisolasi dari latar belakang dan warna lainnya dengan tingkat keberhasilan yang baik.
2. Warna merah memerlukan dua rentang threshold Hue (0–10 dan 160–179) akibat karakteristik wrap-around nilai Hue dalam skala OpenCV, sementara hijau (40–90) dan biru (100–140) masing-masing hanya memerlukan satu rentang.
3. Histogram RGB menunjukkan bahwa distribusi piksel citra tulisan nama didominasi oleh nilai tinggi (putih) yang merepresentasikan latar belakang kertas, dengan distribusi lebih kecil di nilai rendah yang merepresentasikan tinta spidol berwarna.
4. Teknik perbaikan gambar backlight menggunakan operasi pixel linear (`cv2.convertScaleAbs`) berhasil diterapkan namun memiliki keterbatasan yang nyata. Peningkatan kontras ($\alpha=1.7$, $\beta=0$) dan kombinasi brightness+contrast ($\alpha=2$, $\beta=30$) mampu mempertegas outline kepala dan rambut subjek serta meningkatkan detail pada area latar belakang, namun wajah subjek tetap tampak dominan gelap sebagai siluet. Hal ini disebabkan oleh nilai piksel awal pada area wajah yang sangat rendah sehingga operasi linear tidak cukup untuk mengangkatnya secara signifikan. Selain itu, area latar belakang (jendela) mengalami color burn akibat nilai pikselnya yang sudah tinggi semakin diperbesar. Kondisi ini menunjukkan bahwa untuk kasus backlight ekstrem, diperlukan teknik yang lebih adaptif seperti CLAHE (Contrast Limited Adaptive Histogram Equalization) atau gamma correction.
5. OpenCV dan Python terbukti menjadi kombinasi yang powerful dan efisien untuk implementasi teknik-teknik pengolahan citra digital, bahkan untuk pemula sekalipun, berkat antarmuka yang intuitif dan dokumentasi yang lengkap.

DAFTAR PUSTAKA

- Adriansyah, A., & Kurniati, D. (2022). Implementasi Library OpenCV dan Python dalam Pengolahan Citra Digital untuk Deteksi Warna Berbasis HSV. *Jurnal Teknik Komputer AMIK BSI*, 8(1), 45–54.
- Farhan, M. A., & Wibowo, A. (2022). Analisis Segmentasi Citra Digital Menggunakan Metode Thresholding Adaptif pada Sistem Deteksi Objek Berwarna. *Jurnal Informatika dan Rekayasa Elektronik (JIRE)*, 5(2), 113–122.
- Hidayatullah, P. (2021). *OpenCV dengan Python: Panduan Lengkap Pengolahan Citra Digital dan Computer Vision*. Bandung: Informatika Bandung.
- Nugroho, H. A., & Santoso, T. B. (2023). Pengantar Pengolahan Citra Digital: Teori dan Implementasi dengan Python dan OpenCV. Yogyakarta: Graha Ilmu.
- Ramadhan, F. A., & Kusuma, W. A. (2023). Konversi Model Warna RGB ke HSV untuk Segmentasi Citra pada Sistem Inspeksi Visual Berbasis Kamera. *Jurnal Nasional Teknik Elektro dan Teknologi Informasi (JNTETI)*, 12(2), 134–141.
- Setiawan, A. D., & Hendra, H. (2022). Penerapan Teknik Peningkatan Kontras dan Kecerahan pada Perbaikan Citra Backlight Menggunakan OpenCV. *Jurnal TRANSISTOR Elektro dan Informatika*, 7(1), 22–31.
- Wahyudi, E., & Pramono, S. H. (2021). Penggunaan Fungsi cv2.convertScaleAbs untuk Optimasi Citra Gelap Akibat Kondisi Pencahayaan Tidak Merata. *Jurnal Rekayasa Sistem dan Teknologi Informasi (RESTI)*, 5(4), 678–686.
- Wicaksono, B. T., & Arifin, A. Z. (2022). Analisis Histogram Citra Digital sebagai Dasar Praproses dalam Sistem Pengenalan Objek Berbasis Deep Learning. *Jurnal Ilmu Komputer dan Informasi (JIKI)*, 15(2), 89–98.